

HW 4.3: Reproducible Data Analysis

Zhiheng Wang

2026-04-13

Table of contents

1	Busiest Airports Analysis	2
1.1	Overview	2
1.2	Passenger Traffic Summary	2
1.3	Traffic Trends Visualization	2
2	Monte Carlo Numerical Integration	4
2.1	Overview	4
2.2	Small Multiple Visualization	4
3	Planning and Prompting GenAI Tools	6
3.1	Data Context and Plan	6
3.1.1	Plan	6
3.2	Plan-Based GenAI Response	7
3.3	Generic Prompt Response	7
3.4	Comparison of Results	10
4	Reflection	10
	Appendix A: GenAI Usage	11
	Usage 1: Plan-Based Prompt for Calcium Data	11
	Usage 2: Generic Prompt for Calcium Data (Provided by Instructor)	11
	Usage 3: General Assignment Assistance	12
	Appendix B: Code	14

1 Busiest Airports Analysis

1.1 Overview

Air travel has undergone dramatic shifts since the onset of the COVID-19 pandemic in 2020. This section examines annual passenger traffic data from six major international airports—Atlanta (ATL), Frankfurt (FRA), Beijing Daxing (PKX), Tokyo Haneda (HND), Hangzhou (HGH), and London Heathrow (LHR)—spanning from 2020 to 2025. By analyzing both a summary table and a trend plot, we can explore how different airports experienced the pandemic’s impact and subsequent recovery.

1.2 Passenger Traffic Summary

Table 1: Annual Passenger Traffic (Millions) at Six Major International Airports, 2020–2025

Airport	2020	2021	2022	2023	2024	2025
ATL - Atlanta	42.9	75.7	93.7	104.7	108.1	106.3
FRA - Frankfurt	18.8	24.8	48.9	59.4	61.6	63.2
PKX - Beijing Daxing	NA	25.1	NA	44.8	49.4	53.6
HND - Tokyo Haneda	31.1	25.9	50.3	78.7	85.9	91.4
HGH - Hangzhou	28.2	28.2	NA	41.2	48.1	50.5
LHR - London Heathrow	22.1	NA	61.6	79.2	83.9	84.5

Note: Source: Wikipedia. NA indicates the airport was not ranked in the top 50 that year.

Table 1 Looking at the table. Atlanta (ATL) was clearly the busiest, peaking at over 108 million passengers in 2024 before a slight drop in 2025. This makes sense since ATL has always been a major hub. Frankfurt (FRA), on the other hand, started really low at around 18.8 million in 2020 due to European travel bans, but it slowly climbed back to about 63.2 million by 2025. For the Chinese airports, Beijing Daxing (PKX) and Hangzhou (HGH) have some NA values because they dropped out of the top 50 during certain pandemic years. However, their numbers grew fast once domestic travel opened up again. Tokyo Haneda (HND) also had a huge jump, going from just 25.9 million in 2021 to over 91 million in 2025 as Japan reopened its borders. London Heathrow (LHR) followed a pretty similar path, recovering from 22.1 million to 84.5 million.

1.3 Traffic Trends Visualization

Figure 1 The plot shows a clear ‘V-shape’ drop in 2020 and 2021 due to COVID. ATL recovered the fastest, likely because US domestic travel bounced back quickly. The Asian airports like

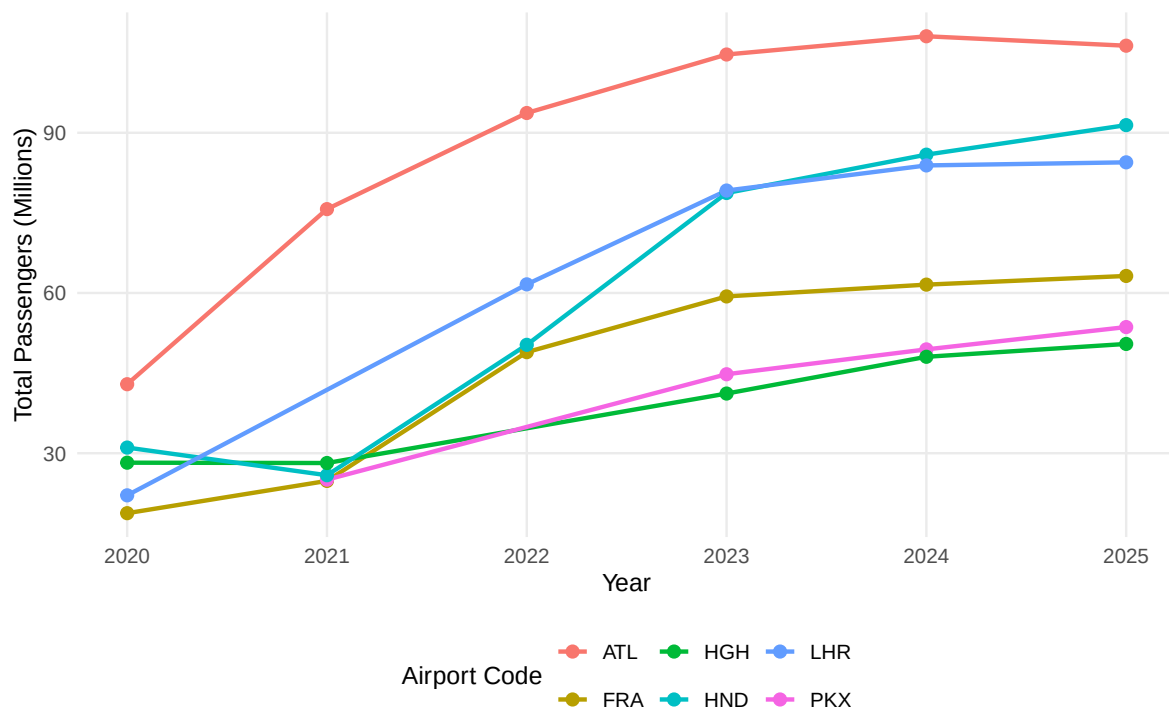


Figure 1: Passenger Traffic Trends at Six Major International Airports, 2020–2025

HND and PKX took longer to recover because of stricter border rules, but you can see a huge spike for HND after 2022

2 Monte Carlo Numerical Integration

2.1 Overview

Monte Carlo methods use random sampling to approximate mathematical quantities that may be difficult to compute analytically. In this section, we apply this technique to estimate the definite integral of the Beta(2, 2) probability density function over the interval $[0, 1]$. The Beta(2, 2) density is defined as:

$$f(x) = \frac{x^{2-1}(1-x)^{2-1}}{B(2, 2)} = 6x(1-x), \quad 0 \leq x \leq 1$$

This is a symmetric, unimodal curve centered at $x = 0.5$ with a maximum value of 1.5. Since $f(x)$ is a proper probability density function, its integral over $[0, 1]$ equals exactly 1. This known true value allows us to evaluate how well the Monte Carlo method performs.

2.2 Small Multiple Visualization

The Monte Carlo method works by generating random points uniformly within a bounding rectangle that encloses the function. In our case, the rectangle spans $x \in [0, 1]$ and $y \in [0, 1.5]$, giving a total rectangle area of 1.5. Each random point is classified as either “On/Below” the curve (green) if $y \leq f(x)$, or “Above” the curve (red) if $y > f(x)$. The estimated integral is then calculated as:

$$\hat{I} = A_{\text{rect}} \times \frac{\text{number of points on/below}}{\text{total number of points}}$$

Figure 2 The plots show how sample size affects the estimate. With only 10 points, the estimate is way off and you can barely see the shape of the curve. By the time we hit 1,000 or 10,000 points, the green and red dots clearly outline the Beta(2,2) curve, and the estimated area gets much closer to the true value of 1. All in all, a larger sample size is necessary for Monte Carlo simulations

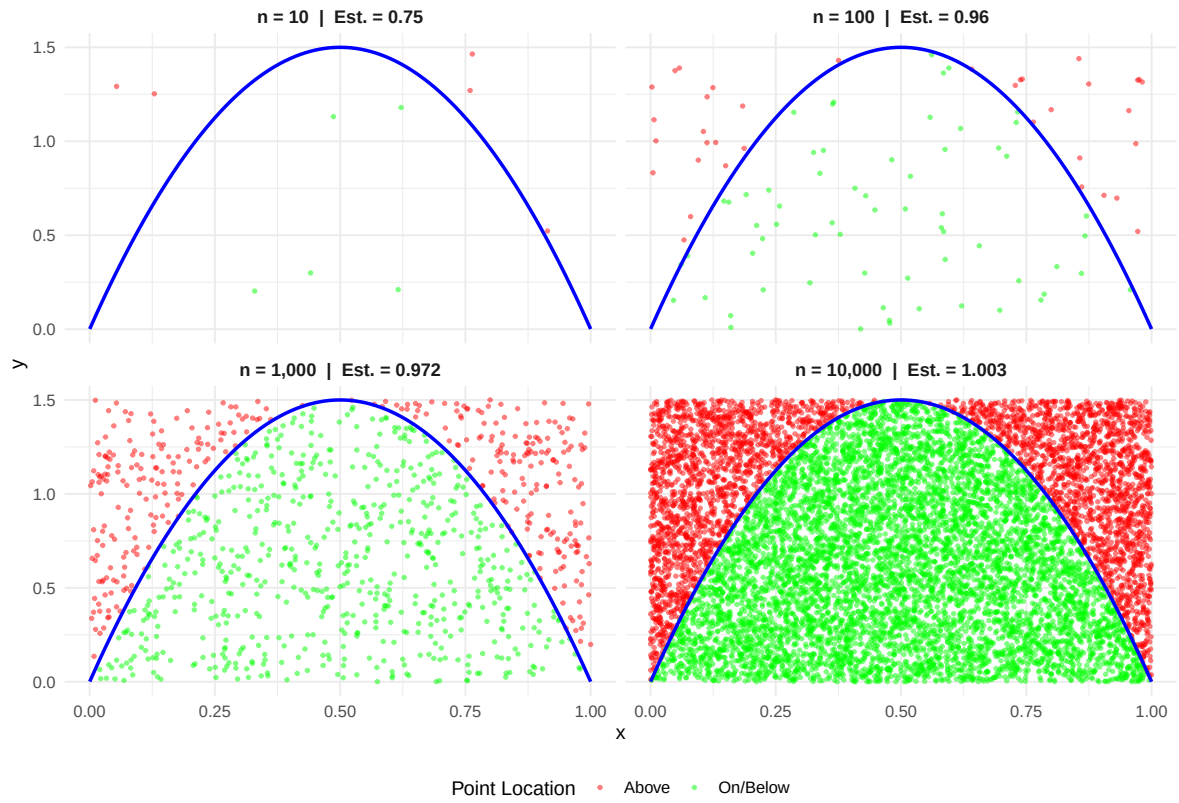


Figure 2: Monte Carlo Estimation of the Beta(2, 2) Integral Across Increasing Sample Sizes

3 Planning and Prompting GenAI Tools

3.1 Data Context and Plan

The calcium data come from a longitudinal study that followed 31 women over four years. At the start of the study, researchers measured the amount of calcium in the ulna from each woman's dominant arm through photon absorptiometry. Fifteen of the women went about their lives as they typically did (the Null Treatment group), while the other 16 women received special help with their diet and completed a regular exercise program (the Diet/Exercise group). Each year, ulnar calcium levels were measured again.

The calcium.csv file contains 8 columns: the first four (`y0`, `y1`, `y2`, `y3`) represent the Null Treatment group's measurements (Initial, Year 1, Year 2, Year 3), and the second four columns (also named `y0`, `y1`, `y2`, `y3`) represent the Diet/Exercise group. These data are *not tidy*—the two groups are stored side by side with duplicated column names, and measurements across time are spread across columns rather than rows.

3.1.1 Plan

We need to take the calcium.csv file, clean it up from its current wide format into a tidy (long) format, and create a line plot to compare how calcium levels change over time between the “Null Treatment” and “Diet/Exercise” groups.

1-Import and Split: First, load up the tidyverse. When we read the data with `read_csv()`, R will automatically handle the duplicate column names (turning them into things like `y0...1` and `y0...5`). We'll just split the data frame right down the middle: columns 1-4 for the Null Treatment group, and columns 5-8 for the Diet/Exercise group.

2-Label and Rename: Next, we need to add a Subject ID column to both tables (1-15 for Null, 1-16 for Diet/Exercise) and a column indicating their treatment group. While we're at it, let's rename those time columns to something actually readable, like “Initial”, “Year 1”, “Year 2”, and “Year 3”.

3-Pivot to Long Format & Clean NAs: We'll use `pivot_longer()` on both data frames to convert them from wide to long format. Since the Null Treatment group has one less person (only 15 subjects), pivoting will create a few empty rows for that missing 16th person—we'll just drop those NA values.

4-Merge and Verify: Stack the two cleaned datasets together using `bind_rows()`. As a quick sanity check, we should end up with exactly 124 rows (15×4 for Null + 16×4 for Diet/Exercise).

5-Plot and Polish: Finally, let's build the visualization using `ggplot2`. We'll put Time on the X-axis and Calcium on the Y-axis, grouping the lines by Subject ID and using colors or facets to separate the two treatment groups. Adding a trend line (like group means) will help highlight

the overall difference over the four years. Throw on a clean theme and some clear labels, and we're good to go.

3.2 Plan-Based GenAI Response

I provided the plan above along with the calcium.csv file to Anthropic Claude and asked it to generate R code following the plan. The tool produced the following code and visualization.

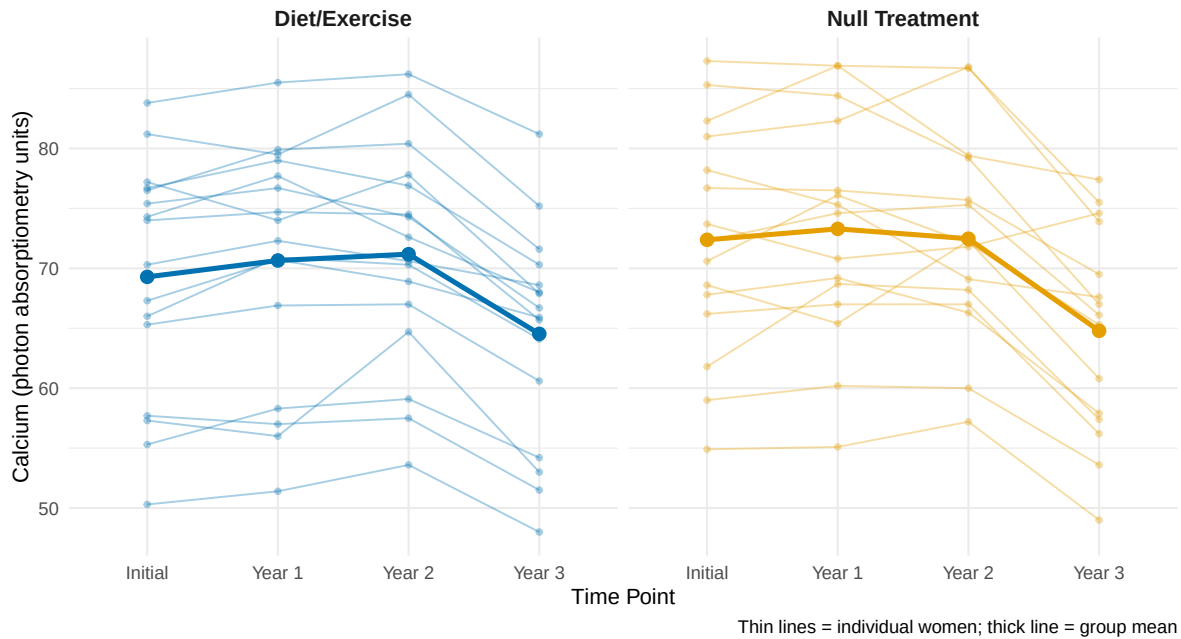


Figure 3: Ulnar Calcium Over Time by Treatment Group (Plan-Based Prompt)

The plan-based prompt produced code that correctly identifies the two treatment groups by their real names (Null Treatment and Diet/Exercise), properly tidies the data using `pivot_longer()`, and creates a faceted line plot showing both individual trajectories and group means. As shown in Figure 3, the visualization clearly distinguishes the two groups and reveals both individual variation and overall trends.

3.3 Generic Prompt Response

The generic prompt provided to Claude was simply: “Create a data visualization that will help me understand these data” along with the calcium.csv file, with no additional context or plan. Figure 4 and Figure 5 show the resulting visualizations.

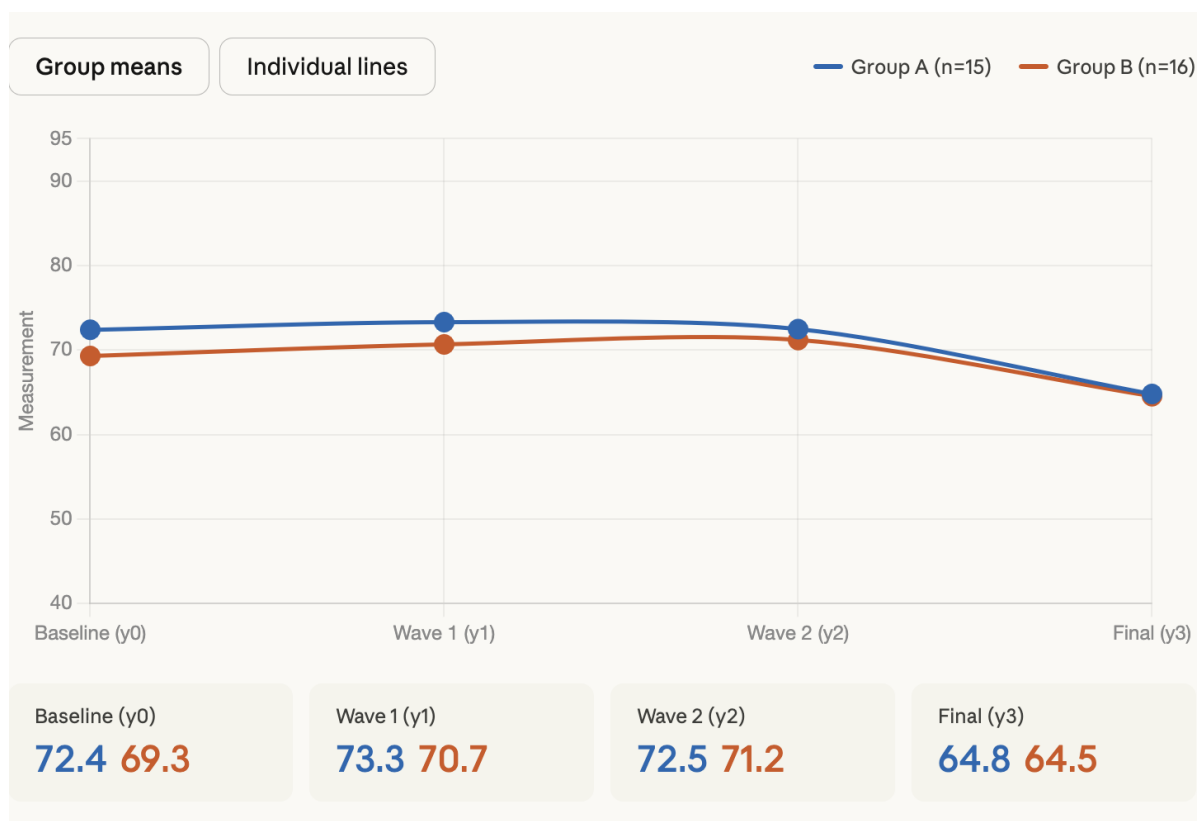


Figure 4: Generic prompt result: group means only

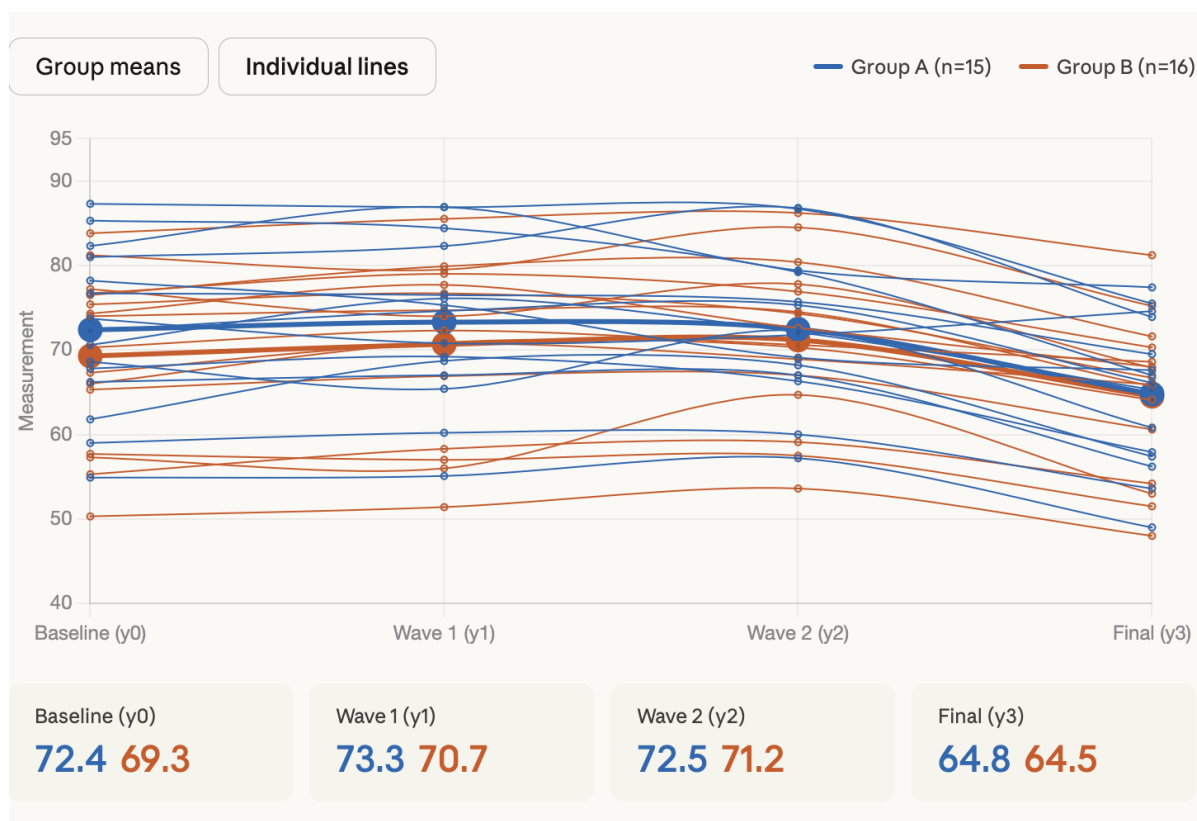


Figure 5: Generic prompt result: individual lines

Claude's generic response labeled the two groups as "Group A" and "Group B" rather than by their actual treatment names, and labeled the time points as "Baseline (y0)," "Wave 1 (y1)," "Wave 2 (y2)," and "Final (y3)." The response text noted that both groups follow the same general pattern of stability followed by a drop at the final measurement, and that Group A starts slightly higher than Group B. It also noted substantial individual variation and the convergence of both groups by the final time point.

3.4 Comparison of Results

The results from the two prompts were completely different. When I gave Claude the detailed plan, it understood the context: it used the correct group names like "Null Treatment" and "Diet/Exercise" and generated clean, reproducible R code with `facet_wrap()` that I could directly use in this Quarto document. On the other hand, when i didn't give the AI the detailed plan, it works really bad, it just named the groups "Group A" and "Group B". Even worse, it gave me an other language plot instead of R code. This just proves that if you want AI to actually help with a coding assignment, you have to tell it exactly what you want it to do step-by-step.

4 Reflection

Looking back at STAT 184, I feel like I've actually gotten a lot more comfortable with R. At the beginning of the semester, dealing with messy data was pretty frustrating. Functions like `pivot_longer()` and `bind_rows()` in the tidyverse used to confuse me, but after doing assignments like the calcium data one, they make a lot more sense now. Also, moving from just plotting basic graphs to actually thinking about how to make them readable (like using `facet_wrap` in the Monte Carlo section) was a big step. Using Quarto has also been cool because it saves me from copying and pasting plots into Word. And, trying out Claude for this assignment showed me that AI is only helpful if you give it exact instructions.

Appendix A: GenAI Usage

Usage 1: Plan-Based Prompt for Calcium Data

Tool: Anthropic Claude (claude.ai, free version)

Date: April 13, 2026

Prompt:

I have a calcium.csv file from a longitudinal study. Here is my plan for tidying and visualizing the data:

[The full plan as written in the “Data Context and Plan” section above was provided, along with the calcium.csv file as an attachment.]

Response:

Claude generated the R code shown in the “Plan-Based GenAI Response” section of this document. The code followed each step of the plan, correctly splitting the groups, renaming columns, pivoting to long format, and creating a faceted line plot with individual trajectories and group means.

Usage 2: Generic Prompt for Calcium Data (Provided by Instructor)

Tool: Anthropic Claude, Sonnet 4.6 Extended

Date: March 31, 2026

Prompt:

Create a data visualization that will help me understand these data. [calcium.csv attached]

Response:

Claude generated an interactive JavaScript-based visualization showing group means and individual lines with toggle buttons. It labeled the groups as “Group A (n=15)” and “Group B (n=16)” and the time points as “Baseline (y0),” “Wave 1 (y1),” “Wave 2 (y2),” and “Final (y3).” The response noted that both groups follow the same pattern of stability followed by a drop at the final measurement, that Group A starts slightly higher, and that there is substantial individual variation.

Usage 3: General Assignment Assistance

Tool: Anthropic Claude, Opus 4.6 (claude.ai)

Date: April 13, 2026

Prompt:

[We need to take the calcium.csv file, clean it up from its current wide format into a tidy (long) format, and create a line plot to compare how calcium levels change over time between the “Null Treatment” and “Diet/Exercise” groups.

1-Import and Split: First, load up the tidyverse. When we read the data with `read_csv()`, R will automatically handle the duplicate column names (turning them into things like `y0...1` and `y0...5`). We'll just split the data frame right down the middle: columns 1-4 for the Null Treatment group, and columns 5-8 for the Diet/Exercise group.

2-Label and Rename: Next, we need to add a Subject ID column to both tables (1-15 for Null, 1-16 for Diet/Exercise) and a column indicating their treatment group. While we're at it, let's rename those time columns to something actually readable, like “Initial”, “Year 1”, “Year 2”, and “Year 3”.

3-Pivot to Long Format & Clean NAs: We'll use `pivot_longer()` on both data frames to convert them from wide to long format. Since the Null Treatment group has one less person (only 15 subjects), pivoting will create a few empty rows for that missing 16th person—we'll just drop those NA values.

4-Merge and Verify: Stack the two cleaned datasets together using `bind_rows()`. As a quick sanity check, we should end up with exactly 124 rows (15×4 for Null + 16×4 for Diet/Exercise).

5-Plot and Polish: Finally, let's build the visualization using `ggplot2`. We'll put Time on the X-axis and Calcium on the Y-axis, grouping the lines by Subject ID and using colors or facets to separate the two treatment groups. Adding a trend line (like `group means`) will help highlight the overall difference over the four years. Throw on a clean theme and some clear labels, and we're good to go.]

Response:

It follows every step in your plan:

Reads the CSV and lets `read_csv()` handle the duplicate column names. Splits into Null Treatment (cols 1–4) and Diet/Exercise (cols 5–8), renames columns to “Initial”, “Year 1”, “Year 2”, “Year 3”, and adds Subject IDs + Treatment labels. Pivots each group to long format with `pivot_longer()`, then combines with `bind_rows()` and drops NAs (the 16th row in the Null group). Verifies the expected 124 rows ($60 + 64$). Creates a faceted line plot where thin

semi-transparent lines show individual trajectories and a thick solid line shows the group mean
— making it easy to compare overall calcium trends between the two treatment groups.

Appendix B: Code

```
# Load libraries
library(tidyverse)
library(kableExtra)

# Create data
airports <- tibble(
  Airport = rep(c(
    "ATL - Atlanta",
    "FRA - Frankfurt",
    "PKX - Beijing Daxing",
    "HND - Tokyo Haneda",
    "HGH - Hangzhou",
    "LHR - London Heathrow"
  ), each = 6),
  Code = rep(c("ATL", "FRA", "PKX", "HND", "HGH", "LHR"), each = 6),
  Year = rep(2020:2025, times = 6),
  Passengers = c(
    42918685, 75704760, 93699630, 104653451, 108067766, 106302208,
    18768601, 24812849, 48918482, 59355389, 61564957, 63189666,
    NA, 25051012, NA, 44786032, 49441029, 53618949,
    31055210, 25876429, 50290705, 78719302, 85900167, 91430000,
    28224342, 28163820, NA, 41170470, 48053915, 50459018,
    22111469, NA, 61614508, 79183364, 83884572, 84463000
  )
)

airports |>
  mutate(Passengers_M = round(Passengers / 1e6, 1)) |>
  select(Airport, Year, Passengers_M) |>
  pivot_wider(
    names_from = Year,
    values_from = Passengers_M
  ) |>
  kable(
    col.names = c("Airport", "2020", "2021", "2022", "2023", "2024", "2025"),
    align = c("l", rep("r", 6)),
    booktabs = TRUE
  ) |>
  kable_styling(
    latex_options = c("hold_position"),
```

```

    full_width = FALSE
  ) |>
  footnote(
    general = "Source: Wikipedia. NA indicates the airport was not ranked in the top 50 that
    general_title = "Note: ",
    footnote_as_chunk = TRUE
  )
airports |>
  filter(!is.na(Passengers)) |>
  ggplot(aes(x = Year, y = Passengers / 1e6, color = Code, group = Code)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = scales::comma_format()) +
  labs(
    x = "Year",
    y = "Total Passengers (Millions)",
    color = "Airport Code"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "bottom",
    panel.grid.minor = element_blank()
  )
# Define the mc_simulate function
mc_simulate <- function(f, n, x_min, x_max, y_min, y_max) {
  x <- runif(n, x_min, x_max)
  y <- runif(n, y_min, y_max)
  df <- data.frame(x = x, y = y)
  df$f_x <- f(df$x)
  df$flag <- ifelse(df$y <= df$f_x, "On/Below", "Above")
  return(df)
}

# Define the Beta(2,2) density function
f <- function(x) { dbeta(x, shape1 = 2, shape2 = 2) }

# Bounding rectangle: x in [0,1], y in [0, 1.5]
rect_area <- (1 - 0) * (1.5 - 0)

# Set seed for reproducibility
set.seed(184)

```

```

# Run simulations for four sample sizes
sample_sizes <- c(10, 100, 1000, 10000)

all_sims <- do.call(rbind, lapply(sample_sizes, function(n) {
  sim <- mc_simulate(f, n, x_min = 0, x_max = 1, y_min = 0, y_max = 1.5)
  p <- mean(sim$flag == "On/Below")
  sim$n_label <- paste0("n = ", format(n, big.mark = ","),
    " | Est. = ", round(rect_area * p, 4))

  sim$sample_size <- n
  sim
}))

# Order the facet labels by sample size
all_sims$n_label <- factor(
  all_sims$n_label,
  levels = unique(all_sims$n_label[order(all_sims$sample_size)])
)

ggplot(all_sims, aes(x = x, y = y, color = flag)) +
  geom_point(alpha = 0.5, size = 0.8) +
  stat_function(
    fun = f,
    color = "blue",
    linewidth = 1
  ) +
  scale_color_manual(
    values = c("Above" = "red", "On/Below" = "green")
  ) +
  facet_wrap(~ n_label, ncol = 2) +
  labs(
    x = "x",
    y = "y",
    color = "Point Location"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "bottom",
    strip.text = element_text(face = "bold", size = 11)
  )

# =====
# Tidying and Visualizing the Calcium Data
# (Code generated by Claude using plan-based prompt)
# =====

```

```

# Step 1: Read the CSV
# Duplicated column names will be auto-renamed by read_csv()
calcium_raw <- read_csv("calcium.csv", show_col_types = FALSE)

# Step 2: Split into two treatment groups
null_wide <- calcium_raw |> select(1:4) # Null Treatment (cols 1-4)
diet_wide <- calcium_raw |> select(5:8) # Diet/Exercise (cols 5-8)

# Step 3: Rename columns consistently and add IDs
time_labels <- c("Initial", "Year 1", "Year 2", "Year 3")

null_wide <- null_wide |>
  set_names(time_labels) |>
  mutate(Subject = row_number(),
         Treatment = "Null Treatment")

diet_wide <- diet_wide |>
  set_names(time_labels) |>
  mutate(Subject = row_number(),
         Treatment = "Diet/Exercise")

# Step 4: Pivot each group to long format
null_long <- null_wide |>
  pivot_longer(cols = all_of(time_labels),
               names_to = "Time",
               values_to = "Calcium")

diet_long <- diet_wide |>
  pivot_longer(cols = all_of(time_labels),
               names_to = "Time",
               values_to = "Calcium")

# Step 5: Combine and clean
calcium_tidy <- bind_rows(null_long, diet_long) |>
  drop_na() |>
  mutate(Time = factor(Time, levels = time_labels))

# Compute group means for the summary layer
group_means <- calcium_tidy |>
  group_by(Treatment, Time) |>
  summarise(MeanCalcium = mean(Calcium), .groups = "drop")
ggplot(calcium_tidy, aes(x = Time, y = Calcium)) +

```

```

# Individual subject trajectories (thin, semi-transparent)
geom_line(aes(group = Subject, colour = Treatment),
          alpha = 0.35, linewidth = 0.5) +
geom_point(aes(group = Subject, colour = Treatment),
           alpha = 0.35, size = 1.2) +
# Group-mean trajectory (thick, solid)
geom_line(data = group_means,
          aes(x = Time, y = MeanCalcium,
              group = Treatment, colour = Treatment),
          linewidth = 1.3) +
geom_point(data = group_means,
           aes(x = Time, y = MeanCalcium, colour = Treatment),
           size = 3) +
# Facet by treatment for clarity
facet_wrap(~ Treatment) +
labs(x      = "Time Point",
     y      = "Calcium (photon absorptiometry units)",
     caption = "Thin lines = individual women; thick line = group mean") +
scale_colour_manual(values = c("Null Treatment" = "#E69F00",
                              "Diet/Exercise"  = "#0072B2")) +
theme_minimal(base_size = 12) +
theme(legend.position = "none",
      strip.text      = element_text(face = "bold", size = 12))

```